

PLATAFORMA DE TELEMEDICINA · MÓDULO 3 · VERSÃO 1.0

Modelagem do Banco e Migrações

Das cinco tabelas escritas em TypeScript até a quarta luz verde no painel: o que é cada tabela e por que ela existe, como o Drizzle transforma código em SQL, como aplicar isso no Supabase sem clicar em nada, e como provar que as regras do banco funcionam.

O que este módulo pressupõe

O Módulo 2 terminou de verdade: `pnpm verificar` dizia “Tudo certo”, `localhost:3000` mostrava três luzes verdes e o projeto está no GitHub sem nenhum `.env`.

O projeto que acompanha este PDF é `telemedicina-modulo-03.zip`. Ele contém **tudo** que este texto cita, no caminho exato citado, e foi testado do zero: instalação, tipos nos 4 pacotes, 8 testes, migrações aplicadas num Postgres limpo com o schema `auth` do Supabase, API e site no ar com quatro luzes verdes.

0. Onde você vai chegar

Ao terminar, `http://localhost:3000` mostra o mesmo painel do Módulo 2 com uma luz a mais:

Luz	O que ela prova
Site (<code>apps/web</code>)	O Next.js subiu e leu <code>apps/web/.env.local</code> .
API (<code>apps/api</code>)	O Fastify subiu na porta 3333 e o site conseguiu chamá-lo.
Supabase (via API)	A API falou com o Supabase usando a chave secreta.
Banco (<code>packages/db</code>) — nova	A API falou com o Postgres pela <code>DATABASE_URL</code> , as 5 tabelas existem e as 2 migrações da pasta <code>drizzle/</code> estão aplicadas.

Quatro luzes verdes = Módulo 3 concluído. O Módulo 4 (segurança por linha: RLS e papéis) começa desse estado.

Os comandos novos, todos rodados na raiz do projeto:

Comando	O que faz
<code>pnpm db:gerar</code>	Compara o schema em TypeScript com a última migração e escreve o próximo arquivo <code>.sql</code> em <code>packages/db/drizzle/</code> .
<code>pnpm db:migrar</code>	Aplica no banco, em ordem, os arquivos <code>.sql</code> que ainda não foram aplicados.
<code>pnpm db:verificar-tabelas</code>	Pergunta ao banco se ele está exatamente como o código espera (também roda dentro do <code>pnpm verificar</code> , passo [6/6]).
<code>pnpm db:studio</code>	Abre um painel local no navegador para olhar (e editar, em dev) as tabelas.

1. Os conceitos, antes de tocar em qualquer arquivo

Se você já sabe o que é chave estrangeira e migração, pule para a seção 2. Se não, leia esta página com calma: tudo que vem depois usa estas seis palavras.

Palavra	O que é (versão de 10 anos)	No nosso projeto
Tabela	Uma planilha com regras: cada coluna tem um tipo (texto, número, data) e o banco recusa o que não encaixa.	perfis, medicos, pacientes, consultas, auditoria
Chave primária (PK)	A coluna que identifica a linha, como o número da carteirinha. Nunca repete.	<code>id</code> em perfis e consultas; <code>perfil_id</code> em medicos e pacientes
Chave estrangeira (FK)	Uma coluna que só aceita valores que existem em outra tabela. “Essa consulta é do médico X” só vale se X existir.	<code>consultas.medico_id</code> aponta para medicos; <code>perfis.id</code> aponta para o login em <code>auth.users</code>

Palavra	O que é (versão de 10 anos)	No nosso projeto
Enum	Uma lista fechada de valores permitidos. Evita “Agendada”, “agendado”, “AGENDADA” convivendo no mesmo campo.	papel (paciente, medico, admin) e status_consulta
Índice	Um sumário que o banco monta para achar linhas rápido. Sem índice ele lê a tabela inteira.	“agenda do médico X hoje” usa consultas_medico_inicio
Migração	Uma receita numerada de mudanças no banco (0000, 0001, 0002...). Aplicada na ordem, recria o banco igual em qualquer lugar.	packages/db/drizzle/*.sql

A regra que organiza o módulo inteiro

O código em packages/db/src/schema/ é a única fonte da verdade sobre o banco. Ninguém cria tabela clicando no painel do Supabase. Se não está numa migração, para o projeto não existe.

Por quê? Tabela criada por clique não tem histórico (quem, quando, por quê), não é reproduzível (o banco de produção fica diferente do de desenvolvimento sem ninguém notar) e não entra no Git — que, em sistema auditável, é parte da prova. Está registrado em docs/adr-0004-migracoes-versionadas.md.

Drizzle é a ferramenta que faz a ponte: você descreve as tabelas em TypeScript (ganha autocompletar e erro de tipo no editor), o **drizzle-kit** transforma isso em SQL (a migração) e o **drizzle-orm** usa o mesmo TypeScript para consultar o banco com tipos. Uma descrição, três usos.

2. O que há de novo no projeto

Tudo que entrou está em packages/db, mais um retoque na API, no site e no verificador. Nada do Módulo 2 foi removido.

```
telemedicina/
|-- packages/db/
|   |-- drizzle.config.ts      diz ao drizzle-kit onde esta o schema, onde gravar
|   |                          as migracoes e de onde ler a DATABASE_URL
|   |-- drizzle/              AS MIGRACOES (vao para o Git)
|   |   |-- 0000_tabelas_iniciais.sql  gerada do schema: enums, 5 tabelas, FKs, indices
|   |   |-- 0001_auth_e_gatilhos.sql   escrita a mao: FK para auth.users + gatilhos
|   |   |-- meta/                      controle interno do Drizzle (nao edite)
|   |-- src/
|   |   |-- schema/                  AS TABELAS, EM TYPESCRIPT (fonte da verdade)
|   |   |   |-- perfis.ts  medicos.ts  pacientes.ts  consultas.ts  auditoria.ts
|   |   |   |-- index.ts   exporta tudo; e o que o drizzle.config le
|   |   |-- ambiente-db.ts      le DATABASE_URL de apps/api/.env (um lugar so)
|   |   |-- estado-banco.ts     "o banco esta como o codigo espera?"
|   |   |-- migrar.ts           pnpm db:migrar
|   |   |-- verificar-tabelas.ts pnpm db:verificar-tabelas (e o [6/6] do verificar)
|   |   |-- schema.test.ts      3 testes que nao precisam de banco
|-- apps/api/src/servidor.ts     GET /saude agora tambem inspeciona o banco
|-- apps/web/src/app/page.tsx   quarta luz: Banco (packages/db)
|-- packages/shared/src/index.ts StatusSaude ganhou o campo "banco"
|-- scripts/verificar-ambiente.mjs passo [6/6]: tabelas e migracoes
|-- docs/adr-0004-migracoes-versionadas.md
`-- docs/adr-0005-auditoria-somente-insercao.md
```

Repare: a senha do banco continua existindo em um único arquivo, apps/api/.env. O drizzle.config.ts, o migrar.ts e a API leem dali. Não há um segundo lugar para colar segredo.

3. As cinco tabelas e por que cada uma existe

Este é o mínimo para um atendimento existir: quem atende, quem é atendido, o encontro entre os dois e o registro de quem mexeu em quê. Prontuário, prescrição, vídeo e pagamento entram nos módulos seguintes, sempre apontando para consultas.

```

auth.users (login, do Supabase)
  | perfis.id = auth.users.id    (migracao 0001)
  v
perfis -----+-----+
  | papel: paciente | medico | admin |
  |                 |         |      |
  |                 |         |      |
  v                 v         v
medicos (CRM, UF)      pacientes (nascimento, CPF)  auditoria (quem fez o que)
  |                 |
  +-----> consultas <-----+
                (inicio, fim, status, motivo)

```

perfis — uma linha por pessoa

Coluna	Tipo	Por quê
id	uuid, PK	É o mesmo id do usuário em auth.users. O Supabase Auth cuida de e-mail e senha; nós guardamos o resto. Não é gerado aqui: vem do login.
papel	enum papel	paciente, medico ou admin. No Módulo 4 é o que decide o que cada um enxerga (RLS).
nome_completo	text	Obrigatório. Aparece em atestado, prescrição e prontuário.
telefone	text	Opcional.
criado_em / atualizado_em	timestampz	Sempre com fuso horário. O atualizado_em é preenchido por um gatilho (migração 0001): ninguém precisa lembrar de atualizá-lo.

medicos — dados profissionais de quem atende

Coluna	Tipo	Por quê
perfil_id	uuid, PK e FK → perfis	Um perfil com papel medico tem exatamente uma linha aqui. Se o perfil for apagado, esta linha vai junto (on delete cascade).
crm + crm_uf	text + char(2), únicos juntos	Resolução CFM 2.314/2022: telemedicina só por médico com registro no CRM. A dupla é única: não existem dois médicos com o mesmo CRM na mesma UF.
especialidade	text	Opcional por enquanto.

pacientes — o mínimo para identificar quem é atendido

Coluna	Tipo	Por quê
perfil_id	uuid, PK e FK → perfis	Mesma ideia de medicos.
data_nascimento	date	Obrigatório: idade muda dose, protocolo e regra de consentimento.
cpf	varchar(11), único	Só os 11 dígitos. É o identificador da prescrição e do atestado. Opcional até o cadastro completo — LGPD art. 6, III (necessidade): não se pede antes de precisar.

consultas — o encontro entre paciente e médico

Coluna	Tipo	Por quê
id	uuid, PK, gerado	gen_random_uuid(): o banco gera; não dá para adivinhar o id da consulta de outra pessoa.

Coluna	Tipo	Por quê
paciente_id / medico_id	uuid, FK → pacientes / medicos	on delete restrict : o banco recusa apagar um médico que tem consultas. Histórico clínico não some por acidente.
inicio / fim	timestampz	Com a regra consultas_fim_depois_do_inicio (CHECK): o banco recusa consulta que termina antes de começar.
status	enum status_consulta	agendada → em_andamento → concluída, ou cancelada. Padrão: agendada.
motivo	text	O que o paciente escreveu ao agendar. Não é o prontuário (Módulo 6).

Dois índices: consultas_medico_inicio e consultas_paciente_inicio. “Minha agenda de hoje” é a pergunta mais frequente do sistema; sem índice, o banco leria a tabela inteira a cada tela.

auditoria — quem fez o que, quando, em qual registro

Coluna	Tipo	Por quê
id	bigserial, PK	Número sequencial. Aqui a ordem importa.
quando	timestampz	Preenchido pelo banco (now()).
quem	uuid, FK → perfis, pode ser nulo	Nulo quando foi o próprio sistema. on delete set null : apagar a pessoa não apaga o rastro.
acao / tabela / registro_id	text	Ex.: prontuario.lido, consultas, <uuid>. Sempre “coisa.acao”.
detalhes	jsonb	O que mudou, se fizer sentido. Nunca dado clínico em claro.
ip	inet	De onde veio a ação.

Por que a auditoria só aceita INSERT

LGPD art. 37 exige o registro das operações de tratamento; CFM 2.314/2022 exige integridade e rastreabilidade. Uma trilha que pode ser editada não prova nada. A tabela nasce agora; a trava (sem UPDATE/DELETE para os papéis da aplicação) entra no Módulo 4 junto com o RLS. Registrado em docs/adr-0005-auditoria-somente-insercao.md.

4. Atualizar o projeto na sua máquina

O zip contém o projeto inteiro no estado do Módulo 3. Ele **não** contém `apps/api/.env` nem `apps/web/.env.local`, então os seus segredos ficam intactos.

1. Feche o `pnpm dev` se estiver rodando (clique no terminal, `Ctrl+C`).
2. Abra o zip. Dentro há uma pasta `telemedicina`; copie o **conteúdo** dela para dentro da sua pasta do projeto, aceitando **substituir** os arquivos existentes. No Windows: selecione tudo dentro de `telemedicina` no zip, arraste para a pasta do projeto, e na pergunta escolha “Substituir os arquivos no destino”.
3. No terminal do VS Code, na raiz do projeto:

```
pnpm install
pnpm typecheck
pnpm test
```

O que você deve ver

`pnpm install` termina com “Done”. Ele baixa duas coisas novas: `drizzle-kit` e o `vitest` do pacote `db`.

`pnpm typecheck` passa nos 4 pacotes sem “error”.

`pnpm test` mostra **dois** arquivos de teste: 3 `passed` em `packages/db` e 5 `passed` em `apps/api`. Os 3 novos conferem que o `schema` e a pasta `drizzle/` combinam — sem precisar de banco.

4. Confira o que mudou aos olhos do Git:

```
git status
```

Deve listar os arquivos novos de `packages/db` e `docs` e os alterados (`README`, `servidor.ts`, `page.tsx`, `index.ts` do `shared`, `verificar-ambiente.mjs`, `package.json`, `pnpm-lock.yaml`). **Não** deve listar nenhum `.env`. Ainda não faça o `commit`: primeiro vamos aplicar e provar.

5. Gerar a migração (e entender por que ela já está lá)

Em um projeto do dia a dia, o fluxo é: você muda o `schema` em TypeScript, roda `pnpm db:gerar`, e o Drizzle escreve o arquivo `.sql` com a diferença. Neste módulo os dois arquivos já vêm gerados e testados no zip, porque o texto precisa citar exatamente o que existe. Então o comando serve como **prova de sincronia**:

```
pnpm db:gerar
```

O que você deve ver

No `schema changes`, `nothing to migrate`

Isso significa: “o que está em `src/schema/` é exatamente o que a última migração descreve”. Se aparecesse um arquivo novo em `drizzle/`, algo no `schema` teria sido alterado sem migração — e o teste 2 de `schema.test.ts` falharia.

5.1 A migração 0000 — gerada do schema

Abra `packages/db/drizzle/0000_tabelas_iniciais.sql`. É SQL puro, na ordem em que o banco precisa: primeiro os dois `enums`, depois as cinco tabelas, depois as chaves estrangeiras, por fim os índices. Cada comando termina com `--> statement-breakpoint`: é o marcador que o Drizzle usa para rodar um comando de cada vez. Compare uma tabela com o arquivo TypeScript correspondente — é a mesma coisa em duas línguas.

5.2 A migração 0001 — escrita à mão

Duas coisas o Drizzle não gera sozinho, e por isso `0001_auth_e_gatilhos.sql` foi criada com `pnpm db:gerar --custom --name auth_e_gatilhos` (que cria um arquivo vazio) e escrita manualmente:

- **A chave para `auth.users`**. O `schema auth` é do Supabase; o Drizzle gerencia só o `public` (`schemaFilter` no `drizzle.config.ts`). Se descrevêssemos `auth.users` em TypeScript, o Drizzle tentaria criá-la — e quebraria o login. Então a `FK perfis.id → auth.users.id` é SQL manual, com `on delete`

cascade: apagou o login, apagou o perfil, e por cascata da 0000, o médico/paciente também.

- **O gatilho de atualizado_em.** A função `definir_atualizado_em()` grava `now()` antes de cada UPDATE em perfis e consultas. Ninguém precisa lembrar de atualizar a coluna: o banco faz.
- **Comentários nas tabelas** (COMMENT ON TABLE). Aparecem no painel do Supabase e em qualquer ferramenta SQL. Documentação que mora dentro do banco.

Três regras sobre migrações

1. **Migração aplicada não se edita.** Errou? A correção é a próxima migração. Editar a 0000 depois de aplicada faz dev e prod divergirem.
2. **Sempre leia o .sql antes de aplicar.** O Drizzle é bom, mas é você quem responde pelo banco. Renomear uma coluna, por exemplo, ele pode entender como “apagar uma e criar outra” — e os dados somem.
3. **Nome descritivo.** `pnpm db:gerar --name telefone_em_perfis` gera `0002_telefone_em_perfis.sql`. Sem `--name`, o Drizzle inventa duas palavras aleatórias.

6. Aplicar no Supabase

Primeiro confirme que a conexão do Módulo 2 continua boa — a senha nova está na DATABASE_URL?

```
pnpm db:testar-conexao
```

Deve responder “Conectado ao Postgres do Supabase”. Se não, a tabela de erros da seção 7.1 do Módulo 2 continua valendo. Agora, o comando central do módulo:

```
pnpm db:migrar
```

O que você deve ver

Aplicando migracoes de packages/db/drizzle ...

2 migracao(oes) aplicada(s).

migracoes no banco: 2 de 2

tabelas em public: auditoria, consultas, medicos, pacientes, perfis

Rode o mesmo comando de novo. Agora ele diz “Nada novo para aplicar: o banco ja estava atualizado.” — o Drizzle guarda o que já rodou na tabela drizzle.__drizzle_migrations. Isso é o que torna o comando seguro para repetir.

Se aparecer...	Significa	Faça
relation "auth.users" does not exist	A DATABASE_URL aponta para um Postgres que não é do Supabase (sem o schema auth)	Confira a URL em apps/api/.env: deve ser a do botão Connect do seu projeto.
... already exists	Já existe uma tabela ou tipo com esse nome — criado por clique no painel, ou de uma tentativa antiga	Apague-o no painel (Table Editor > ... > Delete) e rode de novo. Depois, nunca mais crie tabela por clique.
password authentication failed	Senha errada na DATABASE_URL	A senha nova (trocada no fim do Módulo 2) está na URL? Project Settings > Database > Reset se precisar.
timeout / ECONNREFUSED	Rede ou firewall na porta 5432	Teste na rede do celular; em rede corporativa peça liberação.

6.1 Ver no painel do Supabase (clique a clique)

1. Abra <https://supabase.com>, entre no projeto telemedicina-dev.
2. No menu lateral esquerdo, clique em **Table Editor** (ícone de tabela, o segundo de cima para baixo).
3. No topo da lista de tabelas há um seletor de schema mostrando **public**. Devem aparecer cinco tabelas: auditoria, consultas, medicos, pacientes, perfis. Clique em **perfis**: as colunas são as do arquivo perfis.ts, e passando o mouse no nome da tabela aparece o comentário da migração 0001.
4. Troque o seletor de schema de **public** para **drizzle**. A tabela __drizzle_migrations tem duas linhas: é o registro do que já foi aplicado. Não edite.
5. Ainda no menu lateral, **Database > Triggers**: devem estar lá perfis_atualizado_em e consultas_atualizado_em.

Alternativa sem sair do computador: pnpm db:studio abre <https://local.drizzle.studio> no navegador com as mesmas tabelas. Ele lê a DATABASE_URL de apps/api/.env; feche com Ctrl+C.

7. Provar que as regras do banco funcionam

Uma tabela existir não é o mesmo que ela se defender. Vamos tentar quatro coisas erradas de propósito e ver o banco recusar cada uma. Tudo no painel do Supabase, no **SQL Editor**.

7.1 Criar um usuário de teste (por clique, no Auth)

1. Menu lateral > **Authentication** > aba **Users** > botão **Add user** > **Create new user**.

2. Email: teste@exemplo.com. Password: qualquer coisa. Marque **Auto Confirm User**. Clique em **Create user**.

3. Na lista, clique no usuário. Copie o **UID** (um uuid como a1b2c3d4- . . .). Você vai colar no lugar de **UID_AQUI** abaixo.

7.2 Rodar o roteiro

Menu lateral > **SQL Editor** > **New query**. Cole o bloco abaixo (troque os dois **UID_AQUI**) e clique em **Run** (ou Ctrl+Enter). Rode um bloco de cada vez para ler a resposta de cada um.

```
-- (a) Um perfil ligado ao login criado no Auth. Deve funcionar.
insert into perfis (id, papel, nome_completo)
values ('UID_AQUI', 'medico', 'Dra. Teste');

-- (b) Atualiza o perfil e confere o gatilho: atualizado_em > criado_em? Deve dar "true".
update perfis set telefone = '11999990000' where id = 'UID_AQUI';
select nome_completo, atualizado_em > criado_em as gatilho_funcionou from perfis;

-- (c) Um perfil SEM login. O banco deve RECUSAR (viola perfis_id_auth_users_id_fk).
insert into perfis (id, papel, nome_completo)
values (gen_random_uuid(), 'paciente', 'Sem login');

-- (d) Um papel que nao existe no enum. Deve RECUSAR (invalid input value for enum papel).
insert into perfis (id, papel, nome_completo) values ('UID_AQUI', 'estagiario', 'X');

-- (e) Uma consulta que termina antes de comecar. Precisa de medico e paciente;
-- usamos o mesmo perfil nos dois papeis so para o teste. Deve RECUSAR
-- (viola consultas_fim_depois_do_inicio).
insert into medicos (perfil_id, crm, crm_uf) values ('UID_AQUI', '123456', 'SP');
insert into pacientes (perfil_id, data_nascimento) values ('UID_AQUI', '1990-01-01');
insert into consultas (paciente_id, medico_id, inicio, fim)
values ('UID_AQUI', 'UID_AQUI', now(), now() - interval '1 hour');
```

O que você deve ver

(a) Success. No rows returned. (b) uma linha com gatilho_funcionou = true. (c) erro violates foreign key constraint "perfis_id_auth_users_id_fk". (d) erro invalid input value for enum papel: "estagiario". (e) os dois inserts de médico e paciente passam; o de consulta falha com violates check constraint "consultas_fim_depois_do_inicio".

Cada erro desses é uma regra de negócio que o banco garante **sozinho**, sem depender de a API ou a tela lembrarem. É a última linha de defesa do prontuário.

7.3 Limpar

Volte em **Authentication** > **Users**, clique nos três pontos do usuário de teste > **Delete user**. Depois, no SQL Editor: select count(*) from perfis; deve responder **0**. Apagar o login apagou o perfil, o médico e o paciente por cascata — exatamente o combinado nas migrações. (Se a consulta do item (e) tivesse sido aceita, o on delete restrict teria impedido isso; é o comportamento desejado com dado clínico real.)

8. O verificador ampliado e as quatro luzes

pnpm verificar

O que você deve ver no fim

[6/6] Tabelas e migracoes do banco (Modulo 3)

ok 2 migracao(oes) em packages/db/drizzle/

ok 5 tabelas presentes: perfis, medicos, pacientes, consultas, auditoria

ok 2 migracao(oes) aplicada(s) - igual a pasta drizzle/

ok perfis.id aponta para auth.users.id

ok gatilhos de atualizado_em em perfis e consultas

Tudo certo. Proximo passo: pnpm dev (quatro luzes verdes em http://localhost:3000)

O passo [6/6] delega a conferência para packages/db (que tem a biblioteca postgres); por isso ele só roda depois do pnpm install e com a DATABASE_URL válida. Cada ERRO vem com o comando que resolve — quase sempre pnpm db:migrar.

pnpm dev

Abra http://localhost:3000: quatro luzes. E http://localhost:3333/saude agora inclui:

```
{
  "ok": true,
  "dados": {
    "api": "no_ar",
    "supabase": "conectado",
    "banco": {
      "estado": "migrado",
      "migracoesAplicadas": 2,
      "migracoesEsperadas": 2,
      "tabelas": ["auditoria", "consultas", "medicos", "pacientes", "perfis"]
    },
    "versao": "0.1.0",
    "horario": "..."
  }
}
```

Luz vermelha em Banco, com o texto...	Causa	Faça
A API não conseguiu falar com o Postgres	DATABASE_URL errada, ou rede bloqueando 5432. A API espera 3 s pelo banco e desiste, para não derrubar a luz da API junto.	pnpm db:testar-conexao e siga a dica dele. Depois reinicie o pnpm dev.
O Postgres responde, mas não tem nenhuma tabela	Migrações nunca aplicadas neste banco	pnpm db:migrar
X de 2 migracoes aplicadas	Só a 0000 foi aplicada (ou a pasta drizzle/ tem mais arquivos que o banco)	pnpm db:migrar. Se o banco tiver MAIS migrações que a pasta, falta git pull / reextrair o zip.

Qualidade, como sempre antes de commitar:

```
pnpm typecheck    # 4 pacotes, sem "error"
pnpm test         # 3 passed (db) + 5 passed (api)
```

9. Como será toda mudança de banco daqui para a frente

Este é o ritual. Ele vai se repetir em quase todos os módulos seguintes; vale decorar a ordem.

1. Edite (ou crie) o arquivo em packages/db/src/schema/. Se for tabela nova, exporte-a em schema/index.ts e adicione o nome em TABELAS_ESPERADAS (estado-banco.ts) — o teste 1 lembra você disso.
2. pnpm db:gerar --name o_que_mudou. Abra o .sql gerado e **leia**.
3. pnpm db:migrar no telemedicina-dev. pnpm verificar deve continuar “Tudo certo”.
4. pnpm typecheck e pnpm test.
5. Commit com o schema, a migração e o snapshot juntos: git add packages/db e mensagem feat(db):
6. Em produção (Módulo 8), o mesmo pnpm db:migrar roda apontando para telemedicina-prod — nunca por clique.

Exercício opcional (e reversível)

Adicione em `perfis.ts`, logo abaixo de `telefone: text("teste")`,. Rode `pnpm db:gerar --name teste_coluna` e abra o arquivo `0002_teste_coluna.sql`: uma linha, `ALTER TABLE "perfis" ADD COLUMN "teste" text;`. Você acabou de ver o Drizzle calcular uma diferença.

Agora desfaça **sem aplicar**: remova a linha do `perfis.ts`, apague `drizzle/0002_teste_coluna.sql` e `drizzle/meta/0002_snapshot.json`, e em `drizzle/meta/_journal.json` remova a entrada de `idx 2` (deixe as duas primeiras). `pnpm db:gerar` deve voltar a dizer “No schema changes”. Se tivesse aplicado, o caminho seria outra migração removendo a coluna — nunca apagar o arquivo.

10. O que cada arquivo novo faz

Arquivo	O que faz
packages/db/drizzle.config.ts	Configura o drizzle-kit: dialeto postgresql, schema em src/schema/index.ts, saída em drizzle/, só o schema public, nomes de arquivo com índice (0000_, 0001_...) e a DATABASE_URL lida de apps/api/.env.
packages/db/src/schema/*.ts	As cinco tabelas em TypeScript, com comentário explicando cada coluna e a norma que a justifica. Exportam também os tipos Perfil, NovoPerfil etc., que a API vai usar.
packages/db/drizzle/0000_tabelas_iniciais.sql	Gerada do schema: 2 enums, 5 tabelas, 5 FKs, 6 índices, 1 CHECK.
packages/db/drizzle/0001_auth_e_gatilhos.sql	Manual: FK perfis.id → auth.users.id, função e gatilhos de atualizado_em, comentários das tabelas.
packages/db/drizzle/meta/	Snapshots e journal do Drizzle. É como ele sabe o que já gerou. Vai para o Git; nunca edite à mão (salvo o exercício da seção 9).
packages/db/src/ambiente-db.ts	lerDatabaseUrl() lê apps/api/.env e para com mensagem clara se faltar; explicarErroDeConexao() traduz os erros comuns. Usado por testar-conexao, migrar e verificar-tabelas.
packages/db/src/estado-banco.ts	inspecionarBanco(): lista tabelas do public, conta migrações aplicadas, confere a FK para auth e os gatilhos. resumirEstado(): migrado / faltam_migracoes / sem_tabelas. TABELAS_ESPERADAS: a lista oficial.
packages/db/src/migrar.ts	pnpm db:migrar. Usa o migrator oficial do Drizzle; mostra quantas aplicou e as tabelas resultantes; traduz erros.
packages/db/src/verificar-tabelas.ts	pnpm db:verificar-tabelas. Mesmo formato ok/ERRO do verificador, com o comando que resolve.
packages/db/src/schema.test.ts	3 testes sem banco: schema exporta as tabelas esperadas; cada entrada do journal tem seu .sql; a 0000 cria as 5 tabelas e a 0001 liga a auth.users.
packages/db/src/conexao.ts	Ganhou connect_timeout (10 s) e onnotice silencioso (sem os avisos "já existe, pulando" da segunda migração).
apps/api/src/servidor.ts	Abre uma conexão com o banco ao subir; GET /saude devolve o estado do banco (com 3 s de limite) e fecha a conexão no Ctrl+C.
apps/web/src/app/page.tsx	Quarta luz, com texto que diz o que fazer quando ela está vermelha.
scripts/verificar-ambiente.mjs	Passo [6/6]: conta as migrações da pasta e delega a inspeção do banco ao pacote db.
docs/adr-0004, adr-0005	As duas decisões deste módulo: migrações versionadas; auditoria só com inserção.

11. Git: commit do módulo

```
git status
git add .
git status
```

Confira: nada de .env na lista, e a pasta packages/db/drizzle/ inteira presente (inclusive meta/). Então:

```
git commit -m "feat(db): schema inicial (perfis, medicos, pacientes, consultas, auditoria) e migracoes 0000-0001"
git push
```

A partir de agora, o histórico do Git conta a história do banco: quem criou cada tabela, quando e por quê (o commit aponta para o módulo e a ADR).

12. Checklist do Módulo 3

[] Zip extraído por cima; pnpm install terminou com “Done”

[] pnpm typecheck sem erros; pnpm test mostra 3 passed (db) e 5 passed (api)

- [] `pnpm db:gerar` responde “No schema changes, nothing to migrate”
- [] `pnpm db:migrar` aplicou 2 migrações; rodado de novo, diz que não há nada novo
- [] Table Editor mostra as 5 tabelas em `public` e `__drizzle_migrations` em `drizzle`
- [] Roteiro da seção 7: o gatilho deu `true` e o banco recusou (c), (d) e (e); usuário de teste apagado e `perfis` voltou a 0 linhas
- [] `pnpm verificar` termina em “Tudo certo” com o passo [6/6] todo ok
- [] `pnpm dev`: quatro luzes verdes em `localhost:3000`
- [] Você sabe explicar por que a 0001 foi escrita à mão e por que uma migração aplicada nunca é editada
- [] Você sabe dizer qual regra o banco garante sozinho em cada um dos três erros da seção 7
- [] Commit feito e enviado, sem nenhum `.env`

Próximo: Módulo 4 — Segurança por linha (RLS), papéis e a primeira rota de negócio. Ele parte das quatro luzes verdes: liga o RLS nas cinco tabelas, escreve as políticas (paciente vê só o seu; médico vê os seus atendimentos; auditoria só aceita inserção), cria a rota de cadastro de perfil na API com auditoria automática, e amplia o `pnpm verificar` com um passo [7/7] que prova que o RLS está ligado — no mesmo padrão: cada comando com a saída esperada, tudo testado antes de ser entregue.